基于Ontology的业务知识与Schema联合召回框架

1. Background（背景）

场景描述

1. token
2. 时间

在企业级 Text-to-SQL 应用中，用户（业务人员）的提问往往包含大量的“业务术语”和“隐性逻辑”。

- 用户问法：“请计算上个季度高价值客户的流失率。”
- 数据库现状：数据库中只有 `t_user_log` 表，字段为 `last_login_time`，`total_spend`，`status_code`。
- 缺失的中间层：数据库中不存在名为“高价值客户”或“流失率”的字段。这些定义存在于业务文档（Wiki、需求文档）中（例如：*高价值=消费>1000元，流失=30天未登录*）。

面临瓶颈

- 语义鸿沟（Semantic Gap）：传统的Schema Linking仅关注“问题词汇”与“表列名”的字符或向量匹配。当问题涉及必须经由外部知识推导的逻辑时，传统方法会失效。
- 知识割裂：现有的RAG（检索增强生成）方案通常将“文档检索”和“Schema检索”分开处理。这导致LLM可能召回了正确的Schema，但不知道计算公式；或者召回了公式，但找不到对应的列。

发展现状

目前的SOTA（State-of-the-Art）正在从单纯的 `Text-to-SQL` 向 `Knowledge-Augmented Text-to-SQL` 演进。研究重点已从“如何更好地编码Schema”转向“如何构建一个包含Schema和Domain Knowledge的统一视图”。

---

2. What（做什么）

Goal（目标）

构建一个异构图谱驱动的联合召回系统（Heterogeneous Graph-based Joint Retrieval System）。

- 核心理念：将“数据库结构（Schema）”和“业务逻辑（Knowledge）”视为同一语义空间中的实体，通过图谱（Ontology）将二者显式连接，实现“查词即查表，查表即查逻辑”。

- 科学定义：给定自然语言查询 Q，在一个包含Schema节点 S 和知识节点 K 的异构图谱 $G=(V, E)$ 中，联合检索出一个最优子图 G'，使得 G' 包含生成正确SQL所需的最小完备集（Minimal Context Set）。

Results（结果）

- 量化指标：
  - Recall@K：Schema召回率与知识Snippet召回率的加权平均值提升。
  - EX (Execution Accuracy)：在BIRD等包含大量外部知识推理的数据集上，SQL执行准确率显著提升。
- 关键交付物：
  - 一个能够动态展示“用户Query -> 业务概念 -> 计算公式 -> 物理字段”推理路径的可视化界面。
  - 相比基线（Baseline），大幅减少LLM输入的Token消耗（因为召回更精准，无需把所有文档塞进Prompt）。

---

3. Why（为什么做）

Values（价值）

- 客户价值（体验与准确性）：解决“幻觉”问题。让非技术人员可以使用业务黑话（Domain Jargon）查询数据，而无需了解底层晦涩的字段命名，真正实现数据民主化。
- 效率价值：通过联合召回修剪掉无关的Schema和文档，降低了LLM推理的上下文长度和延迟。
- 维护价值：将业务逻辑从Prompt工程中解耦出来，沉淀为知识图谱/Ontology，形成企业的“指标字典”，实现Single Source of Truth。

Challenges（挑战）

- 异构性挑战：非结构化的文本（如“净利润等于收入减成本”）如何与结构化的图（Table: Finance, Column: Revenue）进行数学上的对齐？
- ~~噪声干扰：单纯的关键词匹配（BM25）容易召回大量无关文档，干扰LLM生成SQL。如何在召回阶段进行“逻辑一致性”校验？~~
- 多跳推理（Multi-hop Reasoning）：用户问的是A（概念），A依赖B（公式），B对应C（字段）。普通向量检索（Dense Retrieval）难以捕捉这种长距离的传递关系。

---

4. How（方法）

本研究提出一种 “分层约束下的联合检索框架” （Hierarchically Constrained Joint Retrieval Framework），核心在于构建一个融合了元模型（Meta-Model）、业务逻辑（Domain

Logic）和物理模式（Physical Schema）的 三层异构图谱（HKSG: Hierarchical Knowledge-Schema Graph）。

模块一：三层图谱构建（Offline Construction）

基于 LPG（Labelled Property Graph） 技术路线，构建 TBox 主导的知识图谱：

- L0：顶层本体层（Upper Ontology Layer）—— [不可变的元约束]
  - 节点定义：预定义世界模型的五大基石实体（`Person`，`Event`，`Org`，`Location`，`Object`）及元属性（如 `Cost`，`Time`，`Status`）。
  - 关系定义：构建全排列的 实体间关系矩阵（Inter-Entity Relation Matrix），如 `Person-Event` 下的 `Participates`（参与）、`Initiates`（发起）等元关系。
  - *作用：为下游知识提供分类模版和推理的“归纳偏置”。*
- L1：业务领域层（Domain Layer）—— [业务逻辑的实例化]
  - 概念实例化（Concept Instantiation）：通过 LLM 从非结构化文档中提取业务实体（如 `VIP Customer`），并通过 `IsA` 边继承自 L0 节点（如 `Person`）。
  - 指标挂载（Metric Attachment）：提取计算逻辑（如 `Order Value`），通过 `IsA` 继承自 L0 的 `Cost` 属性，并挂载于 L1 概念上。
  - 事务关系（Transactional Relations）：实例化业务动作（如 `Places Order`），并将其定义为 L0 元关系（如 `Participates`）的子类。
- L2：物理数据层（Physical Schema Layer）—— [数据的物理落地]
  - Schema 映射：将数据库的 `Table` 和 `Column` 导入图谱。
  - 跨层连接：通过 `MappedTo` 边将 `Table` 连接到 L1 `Concept`；通过 `CalculatedBy` 边将 L1 `Metric` 连接到具体的 `Column` 和 SQL 表达式。
  - 物理支撑：将数据库的 外键约束（Foreign Key） 显式建模为边，作为 L1 业务关系在物理层的支撑路径。

---

mengchuishi1

步骤	任务	核心技术/方法	自动化程度
Step 1	构建 L0	人工设计 (Ontology Engineering)	0% (人工)
Step 2	导入 L2	SQL 解析器 (如 JSqlParser)	100% (脚本)
Step 3	提取 L1	LLM (Information Extraction)	90% (LLM)
Step 4	对齐 L1->L0	LLM (Zero-shot Classification)	95% (LLM)
Step 5	落地 L1->L2	Vector DB + LLM Re-ranking	80% (混合 + 人工审核)

- L0 (顶层)：由 人（专家） 定义，保证绝对的逻辑严谨性。
- L2 (底层)：由 程序（规则） 扫描，保证物理信息的准确性。
- L1 (中间层)：由 LLM（大模型） 负责从非结构化文档中大规模提取，并进行对齐。

第一步：L0 层构建 —— 专家定义 (Expert Defined)

这一层是基础，不能交给 LLM 瞎编，必须人工设定标准。

- 方法：人工编写（Config/JSON/YAML）。
- 来源：基于“知识基准”（人地事物组织 + 关系矩阵）。
- 过程：
  a. 定义 5 个核心节点类（Class）。
  b. 定义 `Cost`，`Time` 等元属性及其约束（如单位、数据类型）。
  c. 预置好“实体间关系矩阵”的所有类型（如 P2P:Service, P2E:Participates）。
- 结果：一个静态的、不可变的图谱骨架。

第二步：L2 层构建 —— 自动扫描 (Automated Scanning)

这一层是物理事实，直接从数据库读取。

- 方法：脚本自动化提取（SQL Parser / Database Metadata API）。
- 来源：数据库的 Information Schema 或 DDL 语句。
- 过程：
  a. Table Extraction：读取所有表名、表注释，创建 `L2_Table` 节点。
  b. Column Extraction：读取列名、类型、注释，创建 `L2_Col` 节点。
  c. Relation Parsing：解析外键约束（Foreign Key），或者解析 Join Log（查询日志），自动建立表之间的连接边。

- 结果：数据库结构的 1:1 镜像图谱。

第三步：L1 层构建 —— LLM 驱动的流水线 (LLM-Driven Pipeline)

这是最核心、最困难的部分。我们需要从海量的业务文档（Wiki、需求文档、指标字典）中提取出 `VIP客户` 、 `下单` 、 `流失率` ，并把它们填入 L0 和 L2 之间。
这个过程分为三个子任务：

3.1 实体与逻辑提取 (Extraction)

- 输入：业务文档（非结构化文本）。
- Prompt 示例：
- "阅读以下文档，提取其中的业务实体（Concept）和计算指标（Metric）。
  文档：'VIP客户是指过去30天消费超过1000元的用户...'
  要求：将提取出的实体映射到 [人, 地, 事, 物, 组织] 中最合适的一个类别。"
- LLM 动作：
  a. 识别出 "VIP客户"。
  b. 判断它属于 "人" (Inherits L0:Person)。
  c. 提取出规则 "消费 > 1000"。

3.2 关系分类 (Classification)

- 输入：提取出的业务行为（如“客户下单”）。
- 任务：将其归类到 L0 的关系矩阵中。
- Prompt 示例：
- "已知实体 'VIP客户'(人) 和 '购买订单'(事)。文档描述了关系 '下单'。
  请从以下 L0 关系中选择父类：[发起, 参与, 监管, 评价...]"
- LLM 动作：将 "下单" 映射为 `Initiates` (发起) 或 `Participates` (参与)。

3.3 物理落地/Schema Linking (Grounding)

- 任务：将 L1 的概念（如“订单金额”）连接到 L2 的物理列（如 `total_amt` ）。
- 方法：Embedding + LLM Rerank。
  a. 向量检索：计算 L1 概念描述的向量，与 L2 表/列注释的向量进行 Top-K 检索。
  b. LLM 判别：
  c. "概念：订单金额（定义：用户实际支付的钱）。
     候选列：1. `order_amt` (订单总额), 2. `discount_amt` (折扣额)。
     请判断哪个列是正确的物理映射？"
- 结果：建立 `CalculatedBy` 边。

模块二：结构化联合检索（Online Retrieval）

当用户输入自然语言查询（Query）时，系统执行以下步骤：

1. 实体链接与意图识别（Entity Linking & Intent Recognition）：

- 识别 Query 中的提及项（Mention），映射到 L1 层的 `Concept` 或 `Metric` 节点作为种子节点（Seed Nodes）。
- *例如：用户问“VIP客户的下单金额”，激活种子节点 `VIP Customer` 和 `Order Value`。*

2. 元路径约束推理（Meta-Path Constrained Reasoning）：
   - 利用 L0 层的结构进行路径剪枝。系统知道 `Order Value` (IsA Cost) 必须依附于 `Event` (IsA Order)，且 `Person` 通过 `Participates` 关系连接 `Event`。
   - 因此，检索算法只会在符合 `Person -> [Participates] -> Event` 模式的路径上游走，自动过滤掉无关的噪声路径。
3. 子图提取与SQL生成（Subgraph Extraction）：
   - 提取包含“业务概念 + 计算公式 + 物理表 + Join路径”的最优子图。
   - 将子图序列化为 Prompt，利用 L2 层的外键信息和 L1 层的公式定义，指导 LLM 生成准确的 SQL。

---

5. How-why：为什么采用“顶层本体驱动”的解法？

Insights（核心洞察）

1. 业务逻辑不是孤立的，它是世界模型的投影：
   - 传统的 Schema Linking 失败，往往是因为它试图直接通过字面相似度匹配“业务黑话”和“物理列名”。
   - 洞察：所有的业务行为（如“下单”、“退货”）本质上都是 L0 层“人参与事”的具体化。如果我们先确定了 Query 涉及 `Person` 和 `Event`，就能利用 L0 的强约束，精准锁定 L1 的业务关系和 L2 的表连接路径。
2. “关系”比“实体”更难检索，需要元模型指引：
   - 实体（如“用户”）容易匹配，但关系（如“谁发起了订单” vs “谁审批了订单”）很难区分。
   - 洞察：通过在 L0 层定义细粒度的关系矩阵（如 `Initiates` vs `Supervises`），我们强制 L1 的业务关系必须继承明确的语义。这使得检索不仅仅是关键词匹配，而是语义类型的匹配。

Advantages（相比传统 RAG/Schema Linking 的优势）

1. 极强的泛化能力（Generalization via L0）：
   - *传统方法*：换一个行业（如从电商换到医疗），Schema Linking 模型通常需要重新微调。
   - *本方法*：L0 层（人、地、事、物、组织）是跨行业通用的。医疗场景下的 `Doctor Treat Patient` 依然符合 `Person [Service] Person` 的 L0 结构。这种结构化先验使得系统在由 L0 驱动的新领域中具有极强的冷启动能力。
2. 精确的逻辑推理与消歧（Precision via Inheritance）：

- *传统方法*：难以区分“金额”是“成本”还是“收入”。
- *本方法*：通过属性继承（如 `Order Value IsA Cost`），系统明确知道该指标具有 `Cost` 的元属性（如货币单位、累加性）。这在生成 SQL 时能避免逻辑错误（如对非数值字段求和）。

3. 检索空间的有效剪枝（Search Space Pruning）：
   - 通过 L0 的约束（例如：`Cost` 属性只能挂载在 `Event` 或 `Object` 上，不能挂载在 `Location` 上），算法在图遍历时可以直接忽略大量不合逻辑的路径，大幅提升检索效率和准确率。
4. 可解释性与溯源（Explainability）：
   - 生成的 SQL 不再是黑盒。系统可以解释：“我选择了 `t_orders` 表，因为它是 `Purchase Order` 的物理映射，而 `Purchase Order` 是 `VIP Customer` 发起的 `Transactional` 事件。”

6. Related Works（相关工作综述）

本研究主要涉及三个领域的交叉：Text-to-SQL中的Schema Linking、检索增强生成（RAG）以及基于图谱的推理。

1. Schema Linking与Schema检索 (Schema Linking & Retrieval)

这一领域主要关注如何从大规模数据库中召回正确的表和列。

- 传统基于嵌入的方法 (Embedding-based Methods):
  早期的工作如 RAT-SQL [ACL 2020] 和 S2SQL 引入了关系感知机制，但在处理海量Schema时效率较低。RESDSQL [AAAI 2023] 提出了“解耦”的思路，设计了一个专门的Schema Filtering模块，利用Cross-Encoder对Schema项进行排序和剪枝。
  - *局限性*：这些方法大多仅基于“自然语言问题”与“列名/注释”的字面或语义匹配，缺乏对“业务逻辑”的理解。如果列名是 `t1.c3` 而问题是“流失率”，这些方法通常会失效。
- 基于图的Schema Linking (Graph-based Linking):BOGIN [EMNLP 2023] 提出了一种基于图神经网络的Schema Linking方法，它构建了一个包含表、列及其关系的异构图，通过GNN来预测Schema项的被提及概率。
  - *局限性*：虽然引入了图结构，但其图节点仅限于数据库内部的元数据（Schema Graph），未包含外部业务知识（Domain Knowledge），因此无法处理需要外部公式推理的复杂查询。

~~2. 知识增强的Text-to-SQL (Knowledge-Augmented Text-to-SQL)~~

随着 BIRD Benchmark [NeurIPS 2024] 的发布，包含外部知识（External Knowledge）的Text-to-SQL成为热点。

- 基于RAG的非结构化检索:
  通用的做法是使用Dense Retrieval（如BM25或SentBERT）检索相关的PDF片段或Wiki描述，将其作为Context拼接给LLM（如 MAC-SQL [COLING 2024] 中的Retriever模块）。

- *局限性*：这种“文本块”形式的知识是非结构化的。LLM经常面临“幻觉”：它可能检索到了知识，但无法准确将其映射到特定的数据库列上（即Text-to-Column的对齐失败）。

- 基于Prompt Engineering的知识注入:
  一些工作尝试在Prompt中硬编码业务规则。
  - *局限性*：受限于Context Window，无法应对大规模企业的复杂知识库，且缺乏可扩展性。

3. 基于Ontology和图谱的联合推理 (Ontology & Graph Reasoning)

这是与研究最贴近，也是目前最具潜力的方向。

- 结构化知识推理:StructGPT [Microsoft] 探索了如何让LLM在结构化数据（KG, Table, DB）上进行迭代推理。Text2Analysis 相关研究中，有学者尝试构建“指标平台（Metric Store）”的中间层，将指标定义为DSL（领域特定语言）。
- 研究定位（Research Gap）：
  虽然图网络在Schema Linking中被使用了（如BOGIN），外部知识在RAG中被使用了（如BIRD基线），但极少有研究将“非结构化的业务文档”转化为“结构化的Ontology”，并将其与“数据库Schema”融合成一张统一的异构图进行联合游走（Joint Walk）。
  大多数现有方案是 Pipeline 式的（先查文档，再查表），而本方法提出的是 Joint Retrieval（联合召回），通过Ontology构建“概念”到“物理列”的显式桥梁，这正是填补了从 Document 到 SQL 之间的 Semantic Structure Gap。

mengchuishi1

方法分类	代表作	核心逻辑	能否处理业务黑话	能否处理隐式逻辑	本方法优势 (Ours)
Dense Retrieval	RESDSQL [AAAI'23]	向量相似度匹配 (Query <-> Schema)	弱 (依赖列名语义)	否	引入Ontology节点作为跳板，解决词汇不匹配
RAG-SQL	BIRD-Baseline	文本块检索 (Query <-> Docs)	能 (召回文本)	弱 (LLM难以将文本对齐到表)	将文本结构化为图，明确知识与列的连接
Graph Linking	BOGIN [EMNLP'23]	GNN在Schema图上传播	弱 (仅限DB内部关系)	否	扩展了图的边界，将Schema图升级为业务语义图
Ours	Ontology-based Joint Retrieval	异构图联合推理 (Schema + Knowledge)	强	强	(本研究核心)

https://gemini.google.com/share/5712516ad335

https://gemini.google.com/share/d879269e7c88

mengchuishi1